Smart Greenhouse Automation System

Project Aim

To develop an automated greenhouse system that monitors and controls temperature, lighting, and ventilation efficiently.

Project Overview

The Smart Greenhouse Automation System uses an Arduino Uno to monitor temperature and light intensity. Based on sensor readings, it automatically controls ventilation, cooling, lighting, and alarms. An LCD displays temperature and system status, while a kill switch provides emergency shutdown, ensuring efficient greenhouse management and plant protection..

Components Used

1. Arduino Uno
2. LDR (Photoresistor)
3. LM35 Temperature Sensor
4. SG90 Servo Motor
5. DC Motor (Fan)
6. NPN Transistor (2N2222/TIP120)
7. 1N4007 Diode
8. White LED
9. Green LED
10. Red LED
11. Piezo Buzzer
12. 16×2 LCD Display
13. 10kΩ Potentiometer
14. Push Button (Kill Switch)
15. 220Ω Resistors (4 Nos.)
16. 1kΩ Resistor
17. 10kΩ Resistor
18. Breadboard
19. Jumper WiresServo OK

20. Buzzer OK
21. Resistors (220Ω)

Construction and Wiring Details

Component	Connection Details
LDR (Photoresistor)	Terminal 1 → 5V, Terminal 2 → A0, 10kΩ resistor connected between A0 and GND
LM35 Temperature Sensor	VCC → 5V, OUT → A1, GND → GND
Servo Motor	Signal → D3, VCC → 5V, GND → GND
DC Motor (Fan)	Terminal 1 → 5V, Terminal 2 → Transistor Collector
NPN Transistor	Base → D5 through 1kΩ resistor, Collector → Motor Terminal 2, Emitter → GND
1N4007 Diode	Connected across motor terminals for back-EMF protection (Cathode → 5V side, Anode → Collector side)
White LED	Anode → D6 through 220Ω resistor, Cathode → GND
Green LED	Anode → D7 through 220Ω resistor, Cathode → GND
Red LED	Anode → D8 through 220Ω resistor, Cathode → GND
Piezo Buzzer	Positive Terminal → D9, Negative Terminal → GND
Kill Switch (Push Button)	One terminal → D2, Other terminal → GND, configured using INPUT_PULLUP
LCD Display (16×2)	RS → D10, E → D11, DB4 → D12, DB5 → D13, DB6 → A2, DB7 → A3, RW → GND, VCC → 5V, GND → GND
10kΩ Potentiometer	One outer pin → 5V, Other outer pin → GND, Middle pin → LCD V0
Power Supply	All components connected to common 5V and GND rails through the breadboard

Component	Arduino Pin(s)	Additional Connections	Purpose

Component		Connection Details	
HC-SR04 Ultrasonic Sensor	TRIG → D2, ECHO → D3	VCC → 5V, GND → GND	Detects vehicles approaching the gate
PIR Motion Sensor	OUT → D4	VCC → 5V, GND → GND	Confirms motion near the gate
LDR (Photoresistor)	A0	Terminal 1 → 5V, Terminal 2 → A0, 10kΩ resistor from A0 → GND	Determines day/night conditions
Servo Motor	Signal → D5	VCC → 5V, GND → GND	Opens and closes the parking gate
Piezo Buzzer	D6	Negative terminal → GND	Provides alerts and warning sounds
Gate Open LED (Green)	D7	220Ω resistor in series, Cathode → GND	Indicates gate is open
Gate Closed LED (Red)	D8	220Ω resistor in series, Cathode → GND	Indicates gate is closed
Ultrasonic Status LED	D9	220Ω resistor in series, Cathode → GND	Indicates ultrasonic sensor is functioning
LDR Status LED	D10	220Ω resistor in series, Cathode → GND	Indicates LDR is functioning
PIR Status LED	D11	220Ω resistor in series, Cathode → GND	Indicates PIR sensor is functioning
Servo Status LED	D12	220Ω resistor in series, Cathode → GND	Indicates servo motor is functioning
Buzzer Status LED	D13	220Ω resistor in series, Cathode → GND	Indicates buzzer is functioning
LCD Display	RS → A1, E → A2, DB4 → A3, DB5 → A4, DB6 →	RW → GND, VCC → 5V, GND →	Displays greetings and

Component		Connection Details	
(16×2)	A5, DB7 → D1	GND	gate status messages
Potentiometer (10kΩ)	V0 (LCD Contrast)	One end → 5V, Other end → GND, Middle pin → LCD V0	Adjusts LCD display contrast

Code Structure / Program Flow

Start

↓

Read Kill Switch

↓

Pressed?

├── Yes → Stop System

│

└── No

↓

Read LDR

↓

Read Temperature

↓

Update LCD

↓

Low Light?

├── Yes → White LED ON

└── No

↓

Temp >30?

├── Fan ON

└── Fan OFF

↓

Temp 30–35?

├── Servo 45°

└── No

↓

Temp >35?

├── Servo 90°

└── No

↓

Temp >40?

├── Alarm Mode

└── Normal Mode

↓

Repeat

Source Code

```
#include <Servo.h>

#include <LiquidCrystal.h>

Servo gate;

// LCD Pins

LiquidCrystal lcd(A1, A2, A3, A4, A5, 1);

// Ultrasonic

const int trigPin = 2;

const int echoPin = 3;

// PIR

const int pirPin = 4;

// Servo

const int servoPin = 5;

// Buzzer

const int buzzer = 6;

// Gate LEDs

const int greenGate = 7;

const int redGate = 8;
```

```
// Health LEDs
const int ultraLED = 9;
const int ldrLED = 10;
const int pirLED = 11;
const int servoLED = 12;
const int buzzerLED = 13;
// LDR
const int ldrPin = A0;
long duration;
float distance;
unsigned long vehicleStartTime = 0;
void setup()
{
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
pinMode(pirPin, INPUT);
pinMode(buzzer, OUTPUT);
pinMode(greenGate, OUTPUT);
pinMode(redGate, OUTPUT);
pinMode(ultraLED, OUTPUT);
  pinMode(ldrLED, OUTPUT);
  pinMode(pirLED, OUTPUT);
  pinMode(servoLED, OUTPUT);
  pinMode(buzzerLED, OUTPUT);
gate.attach(servoPin);
  gate.write(0);
```

```
  lcd.begin(16, 2);
digitalWrite(redGate, HIGH);
lcd.print("SMART PARKING");
  lcd.setCursor(0, 1);
  lcd.print("SYSTEM READY");
delay(3000);
  lcd.clear();
}
void loop()
{
  // Health LEDs
  digitalWrite(ultraLED, HIGH);
  digitalWrite(ldrLED, HIGH);
  digitalWrite(pirLED, HIGH);
  digitalWrite(servoLED, HIGH);
  digitalWrite(buzzerLED, HIGH);
int ldrValue = analogRead(ldrPin);
  bool night = (ldrValue < 500);
// Greeting
  if (night)
  {
    lcd.setCursor(0, 0);
    lcd.print("GOOD EVENING   ");
  }
  else
```

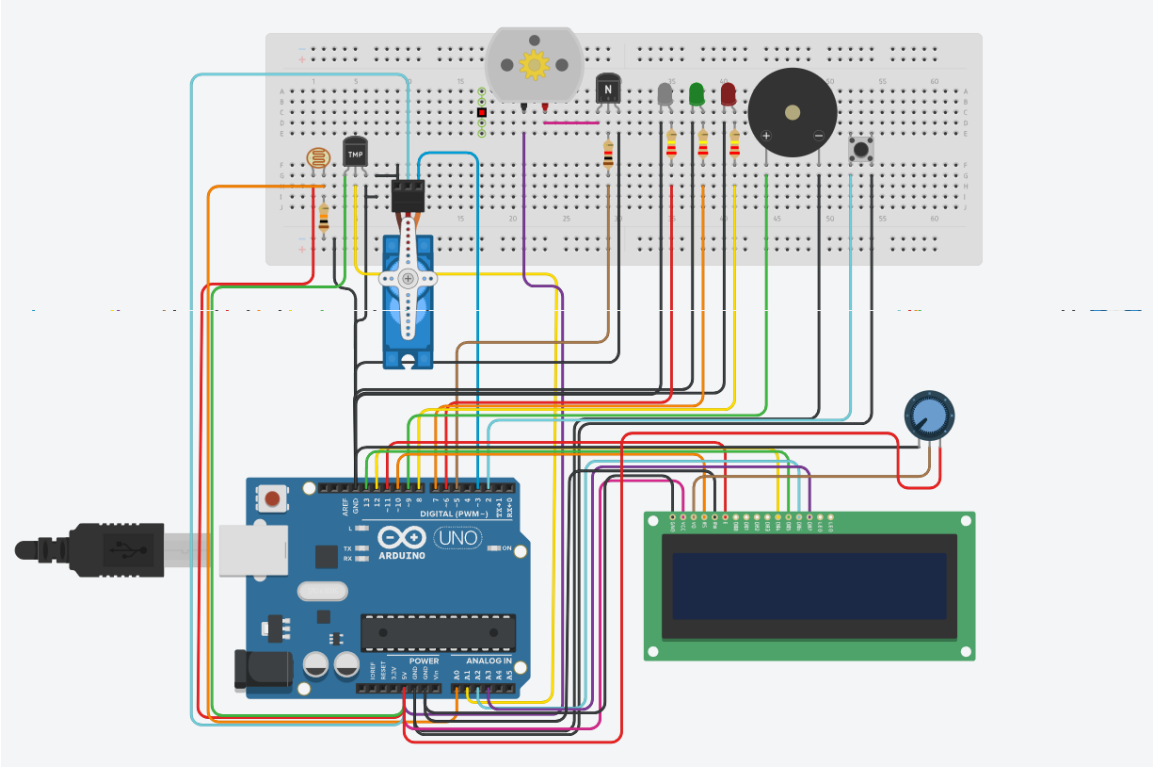
```
 {
   lcd.setCursor(0, 0);
   lcd.print("GOOD MORNING   ");
 }
// Ultrasonic
 digitalWrite(trigPin, LOW);
 delayMicroseconds(2);
digitalWrite(trigPin, HIGH);
 delayMicroseconds(10);
digitalWrite(trigPin, LOW);
duration = pulseIn(echoPin, HIGH);
distance = duration * 0.034 / 2;
bool motion = digitalRead(pirPin);
if (distance <= 20 && motion)
 {
   lcd.setCursor(0, 1);
   lcd.print("VEHICLE FOUND  ");
if (vehicleStartTime == 0)
    vehicleStartTime = millis();
// Night warning
   if (night)
   {
    lcd.clear();
    lcd.print("NIGHT MODE");
tone(buzzer, 1000);
    delay(2000);
```

```
    noTone(buzzer);
  }
openGate();
// Vehicle blocking gate
  while (distance <= 20)
  {
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
digitalWrite(trigPin, LOW);
duration = pulseIn(echoPin, HIGH);
    distance = duration * 0.034 / 2;
if (millis() - vehicleStartTime > 5000)
    {
      lcd.clear();
      lcd.print("REMOVE VEHICLE");
      lcd.setCursor(0, 1);
      lcd.print("GATE BLOCKED");
tone(buzzer, 800);
    }

    if (distance > 20)
    {
      noTone(buzzer);
      vehicleStartTime = 0;
```

```
      break;
    }
   }

   closeGate();
  }
  else
  {
   vehicleStartTime = 0;

   lcd.setCursor(0, 1);
   lcd.print("GATE READY    ");
  }

  delay(200);
}

void openGate()
{
  digitalWrite(redGate, LOW);
  digitalWrite(greenGate, HIGH);

  lcd.clear();
  lcd.print("GATE OPEN");
  lcd.setCursor(0, 1);
  lcd.print("WELCOME");
```

```
  gate.write(90);

  delay(3000);
}

void closeGate()
{
  lcd.clear();
  lcd.print("GATE CLOSING");
  lcd.setCursor(0, 1);
  lcd.print("PLEASE WAIT");

  delay(1000);

  gate.write(0);

  digitalWrite(greenGate, LOW);
  digitalWrite(redGate, HIGH);

  lcd.clear();
}.
```

Project Photograph



Links

Google drive : https://drive.google.com/drive/folders/1GEi2cqGkShexGLSm9h5sOYaS0CmqDtWS?usp=drive_link

TinkerCad : https://www.tinkercad.com/things/6J7J2iu2PiZ/editel?returnTo=%2Fdashboard&sharecode=DAf60AfZr-BKVl7YNNOqVloo_Jb3vxt9bV7eLmKw5Ho

Git hub repo : https://github.com/AmanAZ5950/CIRCUITRON.git